W-Ranked Diagonal Hyperreflection

A Computable Contender 20 for BusyLean

Research note prepared for Vladimir Reshetnikov

October 13, 2026

Abstract

The current BusyLean record, Contender 19, combines a large-Veblen-length ladder of universes with a natural-number-indexed hierarchy of diagonal transformer lifts. This paper proposes and formally verifies a qualitative extension: replace the finite rank by an arbitrary well-founded `WTree Nat`. At a rank-tree node, all child-rank lifts are spliced into one lower lift; that lower lift both pre-amplifies the base transformer and serves as the local lift while a fixed schedule is traversed. The resulting operation, `wDiagonalLift`, is structurally recursive and therefore computable.

The new number is

$$C_{20} = F_{20}(F_{20}(42)),$$

where F_{20} applies W-ranked hyperreflection to the large-Veblen ladder, uses the entire Contender 19 construction as its oracle seed, and chooses both its schedule and its W-valued rank by autonomous hyper-closures above the Contender 19 schedule. Lean proves the pointwise inequality

$$\mathrm{family}_{19}(n) < F_{20}(n)$$

and, in particular, the required comparison

$$C_{19} < C_{20}.$$

The checked development also proves strict parent-over-child rank growth and a program domination theorem for every enumerated large-Veblen-ladder program at the relevant depth.

The development was checked on Lean 4.34.0. With Contender 19 precompiled, the new definition phase took 0.84 seconds and the new comparison proof phase took 0.35 seconds on the recorded GitHub Actions runner. The focused audit reports only the repository-permitted axioms `propext` and `Quot.sound`; the transitive closure of `contender_20` avoids all earlier contender constants. No attempt is made to evaluate the numeral.

Contents

1 Problem statement and standard of success **3**

2 The audited baseline: Contender 19 **3**

2.1 The large-Veblen ladder 3

2.2 Finite diagonal hyperreflection 4

3 The design gap: from finite rank to well-founded rank **4**

4 Abstract transformer framework **4**

4.1 Families, transformers, and lifts 4

4.2 Why monotonicity is deliberately absent 5

5 W-ranked diagonal hyperreflection **5**
5.1 Splicing a family of lifts . . . 5
5.2 The recursive definition . . . 6
5.3 Computability . . . 6

6 Formal growth theorems **6**

7 Definition of Contender 20 **7**
7.1 Seed: the whole Contender 19 family . . . 7
7.2 A schedule and a W-valued rank . . . 7
7.3 The numerical function and final diagonal . . . 8

8 Dominance over Contender 19 **8**
8.1 Pointwise family domination . . . 8
8.2 Program domination . . . 8
8.3 The required comparison . . . 9

9 Why the margin is structurally significant **9**
9.1 Exact formal margin . . . 9
9.2 Heuristic growth calibration . . . 9

10 Rule compliance and audit **10**
10.1 Closed, constructive natural number . . . 10
10.2 No direct dependence on prior constants . . . 10
10.3 Axiom report . . . 10

11 Build validation and timing **10**

12 Repository integration **11**

13 Limitations and research integrity **11**

14 Further research questions **12**

15 Conclusion **12**

A Lean theorem inventory **13**

B Core source excerpt **13**

C Validation receipt summary **14**

1 Problem statement and standard of success

BusyLean is a formal version of the "name the biggest number" game: a submission must define a closed natural number, must be computable in principle, and must include a kernel-checked strict comparison with the previous accepted contender. Its rules additionally exclude `noncomputable`, `partial`, `unsafe`, opaque stand-ins, external implementations, and unapproved axioms. A new contender should use the previous *construction*, rather than merely inserting the previous constant into a trivial successor expression.

For this project, the formal success criterion is therefore not an informal claim about a fast-growing hierarchy. It is the existence of Lean terms

```
def contender_20 : Nat := ...

theorem contender_19_lt_contender_20 :
  contender_19 < contender_20 := ...
```

that compile under the repository's pinned toolchain and pass its axiom and dependency discipline. The stronger goal pursued here is to make the new construction qualitatively different from Contender 19, not merely to enlarge an iteration parameter.

The exact formal claims established in the accompanying source are:

(i) W-ranked diagonal hyperreflection preserves progressivity;

(ii) every W-rank is strictly inflationary over its base transformer;

(iii) every visible child rank is strictly below its parent rank;

(iv) Contender 19's entire family is pointwise below F_{20};

(v) every bounded large-Veblen-ladder program over the new seed is below F_{20};

(vi) $C_{19} < C_{20}$.

2 The audited baseline: Contender 19

2.1 The large-Veblen ladder

Contender 19's reflected object language is a ladder of universes whose atoms are large-Veblen terms. In the repository implementation these are normal Schuette-style bracket symbols: a constructor $\varphi(A)$ carries a sorted array of position–value pairs, and positions are themselves terms. The repository proves well-foundedness of the comparison and instantiates the generic ladder normalization theorem with that well-founded atom order.

The numerical reflection operator is represented by a transformer `lTransformer`. Given a countable oracle family f_i, the primitive reflection value exceeds:

- each visible oracle value $f_i(n)$, for $i \leq n$; and
- each enumerated, well-typed large-Veblen-ladder program of depth at most n, run with the packed oracle family.

This is the core source of language strength inherited by the present work.

2.2 Finite diagonal hyperreflection

The generic amplifier used by Contender 19 is the finite hierarchy `diagonalLift`. In mathematical pseudocode, for a fixed schedule $S \in \mathsf{WTree}(\mathbb{N})$,

$$D_S(0)(B) = \mathrm{liftTransformer}(B),$$
$$D_S(k+1)(B) = \mathrm{stageLift}\big(D_S(k), D_S(k)(B), S\big).$$

Thus the rank-k lift occurs twice at successor rank: once as the local lifting operation used throughout the schedule, and once as a pre-amplifier of the base transformer.

Contender 19 chooses a self-fed finite rank and a self-fed autonomous schedule:

```
def family18 (n : Nat) : Nat := F18 (F18 n)

def schedule19 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family18 n + 42)
      (Autonomous.encode tree12))

def rank19 (n : Nat) : Nat := family18 n + 1

def F19 (n : Nat) : Nat :=
  diagonalLift (schedule19 n) (rank19 n)
    lTransformer (fun _ => family18) n n

def contender_19 : Nat := F19 (F19 42)
```

This is already an extremely strong construction. A successful successor should not merely replace `rank19 n` by a larger natural number.

3 The design gap: from finite rank to well-founded rank

Natural-number rank is a genuine limitation in the *shape* of the amplifier. No matter how enormous k is, $D_S(k)$ is generated by a single linear successor recursion. The rank carries no branching data and no limit structure. By contrast, $\mathsf{WTree}(\mathbb{N})$ supports arbitrary countable branching at every well-founded node. It can therefore serve as executable syntax for a transfinite recursion without introducing a nonconstructive ordinal type or an oracle.

The key design question is how a node with countably many child ranks should combine them. A maximum is unavailable: the child objects are transformer lifts, not numbers, and reflection is not monotone in its oracle. The correct operation is instead the same explicit-coordinate splice already used at the transformer level. Output coordinate i selects child i. This is important: every growth proof can point to a concrete occurrence of a child, rather than appeal to a false monotonicity principle.

The proposed jump is therefore:

$$\text{finite linear rank } k \in \mathbb{N} \quad \longrightarrow \quad \text{arbitrary well-founded rank } R \in \mathsf{WTree}(\mathbb{N}).$$

The object language remains the strongest language currently implemented in BusyLean, so the new work isolates the improvement in the amplifier itself.

4 Abstract transformer framework

4.1 Families, transformers, and lifts

A BusyLean oracle family is extensionally a function

$$\mathsf{Family} = \mathbb{N} \to \mathbb{N} \to \mathbb{N}.$$

The first argument is a family coordinate and the second is the numerical input. A transformer is a map between such families,

$$\mathsf{Transformer} = \mathsf{Family} \to \mathsf{Family},$$

and a transformer lift acts one type level higher,

$$\mathsf{Lift} = \mathsf{Transformer} \to \mathsf{Transformer}.$$

A transformer T is *progressive* when every visible seed coordinate is strictly below every output coordinate at the same numerical input:

$$\mathrm{Progressive}(T) :\Longleftrightarrow \forall f, i, o, n,\ i \le n \Rightarrow f(i,n) < T(f)(o,n).$$

A lift L is *strictly inflationary* when, for every progressive base transformer T,

$$T(f)(i,n) < L(T)(f)(o,n)$$

for all visible $i \le n$ and all output coordinates o.

4.2 Why monotonicity is deliberately absent

It would be tempting to order oracle families pointwise and prove reflection monotone. That statement is generally false: a reflected program may inspect an oracle value and branch on it, so increasing the oracle can decrease the program's result. BusyLean's existing proofs correctly avoid this trap. Every comparison follows an explicit occurrence of a seed value, a selected child, or a selected transformer coordinate.

The W-ranked development follows the same discipline. This makes its core proofs robust and explains the apparently asymmetric use of output coordinate 0 in splice operations.

5 W-ranked diagonal hyperreflection

5.1 Splicing a family of lifts

Given a countable family L_i of transformer lifts, define

$$\mathrm{spliceLifts}(L)(B) = \mathrm{spliceTransformers}\big(i \mapsto L_i(B)\big).$$

The Lean definition is one line:

```
def spliceLifts (lifts : Nat -> TransformerLift) :
    TransformerLift :=
  fun base => spliceTransformers (fun i => lifts i base)
```

At output coordinate i, the resulting transformer runs the transformed operator selected by i, at that operator's canonical output coordinate 0.

Lemma 5.1 (Splice preservation)**.** *If every L_i preserves progressivity, then* spliceLifts(L) *preserves progressivity. If every L_i is strictly inflationary, then the splice is strictly inflationary.*

Proof idea. For progressivity, output coordinate o selects $L_o(B)$, and the assumed progressivity of that transformed base supplies the inequality. For strict inflation, use the strictness theorem for L_o and its canonical output coordinate 0. No relation between distinct components is required. □

5.2 The recursive definition

Fix a local schedule $S \in \mathsf{WTree}(\mathbb{N})$. The W-ranked hierarchy $W_S(R)$ is defined by structural recursion on the rank tree R:

$$W_S(\mathsf{leaf}) = \text{liftTransformer},$$
$$W_S(\mathsf{node}(f))(B) = \text{stageLift}(L, L(B), S),$$

where

$$L = \text{spliceLifts}(i \mapsto W_S(f(i))).$$

In Lean:

```
def wDiagonalLift (schedule : WTree Nat) :
    WTree Nat -> TransformerLift
  | .leaf => liftTransformer
  | .node f =>
      let lower : TransformerLift :=
        spliceLifts
          (fun i => wDiagonalLift schedule (f i))
      fun base =>
        stageLift lower (lower base) schedule
```

This is the exact W-tree analogue of one successor step of finite diagonal hyperreflection. All child-rank lifts are exposed through `lower`; then `lower` occurs both intensionally in the pre-amplified base and extensionally as the lift applied at every schedule node.

5.3 Computability

Proposition 5.2 (Constructivity)**.** *For every finite Lean value S, R : `WTree Nat`, the term `wDiagonalLift S R` is a computable transformer lift in the ordinary operational semantics of Lean.*

Proof. The only recursion defining `wDiagonalLift` is structural recursion on R. The leaf branch returns an existing computable lift. The node branch forms ordinary Lean functions, uses the existing structural `stageLift`, and recursively calls `wDiagonalLift` only on immediate subtrees $f(i)$. No choice operator, unbounded search, partial definition, unsafe code, opaque numerical stand-in, or external primitive occurs. □

The proposition says "computable in principle," not "feasible to normalize." As with the earlier contenders, evaluating the final numeral is both unnecessary and practically impossible.

6 Formal growth theorems

The accompanying file `TreeHyperReflectionFacts.lean` establishes three central theorems.

Theorem 6.1 (Progressivity at every W-rank)**.** *For every schedule S and rank tree R, the lift $W_S(R)$ preserves progressivity.*

Induction structure. At a leaf this is the existing theorem for `liftTransformer`. At a node, the induction hypotheses for all children imply progressivity of their splice. That splice sends a progressive base to a progressive transformed base. The existing `stageLift_progressive` theorem then closes the node case. □

Theorem 6.2 (Strict inflation at every W-rank)**.** *For every schedule S and rank tree R, the lift $W_S(R)$ is strictly inflationary.*

Induction structure. The leaf case is the existing strictness theorem for autonomous lifting. At a node, let L be the splice of child-rank lifts. By induction and the splice lemmas, L preserves progressivity and is strictly inflationary. Hence

$$B(f)(i,n) < L(B)(f)(i,n).$$

The generic `base_lt_stageLift` theorem then gives

$$L(B)(f)(i,n) < \mathrm{stageLift}(L, L(B), S)(f)(o,n).$$

Transitivity yields the result. □

Theorem 6.3 (Child-rank strictness)**.** *Let $R = \mathsf{node}(f)$. For every progressive base transformer B, seed family g, and visible child coordinate $i \le n$,*

$$W_S(f(i))(B)(g)(0,n) < W_S(R)(B)(g)(o,n)$$

for every output coordinate o.

Proof idea. Coordinate i of the splice is definitionally the child transformer's canonical output coordinate. The parent stage strictly exceeds the spliced lower transformer. This identifies a concrete child occurrence and again requires no monotonicity. □

Child-rank strictness is the formal reason the W-tree is not a decorative parameter. Moving from a visible child to its parent forces strict numerical growth at the same input.

7 Definition of Contender 20

7.1 Seed: the whole Contender 19 family

Define

```
def family19 (n : Nat) : Nat := F19 (F19 n)
```

so that `family19 42` unfolds to `contender_19`. This uses the previous construction as a function of arbitrary input; it does not mention the previous contender constant.

7.2 A schedule and a W-valued rank

The repository's `Autonomous` syntax supplies computable operations on well-founded trees: encoding, realization, fixed-point-like closures, and higher hyper-closures. They are used here as transparent tree syntax, not as a quotient with asserted ordinal identities.

First place a self-fed autonomous hyper-closure above Contender 19's schedule:

```
def schedule20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family19 n + 42)
      (Autonomous.encode (schedule19 n)))
```

Then place another such closure above the new schedule, retaining the result as a tree rather than collapsing it to a natural-number rank:

```
def rank20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family19 n + 42)
      (Autonomous.encode (schedule20 n)))
```

The two roles are intentionally distinct. `schedule20 n` controls the operator-stage traversal at each rank node. `rank20 n` controls the transfinite shape of the hierarchy of lifts itself.

7.3 The numerical function and final diagonal

```
def F20 (n : Nat) : Nat :=
  wDiagonalLift (schedule20 n) (rank20 n)
    lTransformer (fun _ => family19) n n

def contender_20 : Nat := F20 (F20 42)
```

Figure 1 shows the dependency structure.

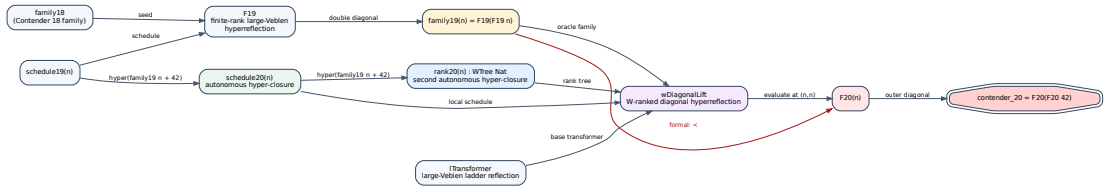


Figure 1: Construction of `contender_20`. The red edge is the central pointwise comparison proved in Lean.

8 Dominance over Contender 19

8.1 Pointwise family domination

Theorem 8.1 (Checked in Lean)**.** *For every $n \in \mathbb{N}$,*

$$\mathrm{family}_{19}(n) < F_{20}(n).$$

Proof. The base transformer `lTransformer` is progressive. By W-rank progressivity, the transformed operator

$$W_{\mathrm{schedule}_{20}(n)}(\mathrm{rank}_{20}(n))(\texttt{lTransformer})$$

is progressive. Apply its progressivity to seed coordinate 0, output coordinate n, and input n. The seed family is constant in its first coordinate, so seed coordinate 0 is exactly $\mathrm{family}_{19}(n)$. The output is definitionally $F_{20}(n)$. □

This pointwise theorem is stronger and more reusable than a comparison only at 42. It certifies that the new construction absorbs the whole previous family at every input.

8.2 Program domination

Let t be an enumerated large-Veblen-ladder program of depth at most n. The primitive reflection theorem gives

$$\mathrm{lvalue}(\mathrm{packFamily}(\mathrm{family}_{19}), n, t) < \mathrm{reflectLFamily}(\mathrm{family}_{19}, n).$$

Strict inflation of the W-ranked lift then gives

$$\mathrm{reflectLFamily}(\mathrm{family}_{19}, n) < F_{20}(n).$$

Lean combines them as lvalue_lt_F20. Thus the new number is not only larger than the previous numerical seed; it also dominates every program in the same bounded large-Veblen ladder when run over the stronger seed.

8.3 The required comparison

The final comparison is short because the general theorems carry the work:

```
theorem contender_19_lt_contender_20 :
    contender_19 < contender_20 :=
  Nat.lt_trans (family19_lt_F20 42)
    (input_lt_F20 (F20 42))
```

Unfolding the definitions, the first inequality is

$$C_{19} = \text{family}_{19}(42) < F_{20}(42),$$

and the second is

$$F_{20}(42) < F_{20}(F_{20}(42)) = C_{20}.$$

Therefore $C_{19} < C_{20}$, as required.

9 Why the margin is structurally significant

There are two distinct senses of "larger" in a formal large-number contest. The first is the exact theorem $C_{19} < C_{20}$. The second is the structural strength of the definition. The present proposal improves both.

9.1 Exact formal margin

The following claims are kernel-checked, not heuristic:

- the whole family `family19` is pointwise below `F20`;
- every W-rank is strictly inflationary over the same base;
- every visible child rank is strictly below its parent;
- every bounded large-Veblen-ladder program over the new seed is below `F20`;
- the final double diagonal strictly exceeds Contender 19.

9.2 Heuristic growth calibration

Any precise fast-growing-hierarchy calibration would require a semantic analysis not present in BusyLean and not claimed here. Nevertheless, the construction supports a robust qualitative interpretation.

Finite `diagonalLift` iterates a rank-successor operation along $\mathbb{N}$. Its self-fed rank may be inconceivably large, but its recursion scheme is still finite and linear. W-ranked hyperreflection instead permits countable branching and well-founded limit structure in the *rank of the lift hierarchy itself*. At a node, all child ranks remain separately addressable through a splice, and the parent strictly exceeds every visible child. In ordinal-language shorthand, the change is analogous to replacing a finite rank closure by a transfinite closure indexed by a concrete well-founded tree.

The rank tree is not chosen near the bottom of the hierarchy. It is a second autonomous hyper-closure above a first new closure above Contender 19's own schedule, and the hyper-iteration count is fed by `family19 n`. This is why the proposal should be regarded as a new amplifier level rather than a parameter tweak.

The word "analogous" is essential. The Lean theorem proves numerical inequalities and recursion properties. It does not identify the order type of `rank20 n`, prove that

the autonomous syntax realizes a specific named ordinal, or equate `F20` with a standard fast-growing hierarchy.

10 Rule compliance and audit

10.1 Closed, constructive natural number

`contender_20` has type `Nat` and is closed. Every new recursive definition is structural:

- `wDiagonalLift` recurses on `WTree Nat`;
- the inherited autonomous operations recurse on their inductive tree syntax or on `Nat`;
- `F20` and `contender_20` are ordinary total definitions.

The new files contain no `noncomputable`, `partial`, `unsafe`, `opaque`, `implemented_by`, `extern`, custom axiom, or native decision procedure.

10.2 No direct dependence on prior constants

A focused closure audit computes all constants reachable through the type and value of `contender_20`. On the validated commit it reports:

```
BusyLean.contender_20: closure of 4186 constants avoids all 15 forbidden names
```

where the forbidden list is `contender_5` through `contender_19`. The definition does use `F19` through `family19`; this is precisely the permitted use of the previous construction as a functional seed.

10.3 Axiom report

The focused audit reports:

Declaration	Reported axioms
`spliceLifts`	none
`wDiagonalLift` and its generic growth theorems	`propext`
`family19`, `F20`, `contender_20`	`propext`, `Quot.sound`
contender_19_lt_contender_20	`propext`, `Quot.sound`

These are exactly within BusyLean's stated allowlist. The appearance of `Quot.sound` is inherited from the accepted normalization stack.

11 Build validation and timing

The attached receipt records a successful GitHub Actions run on Ubuntu 24.04 with Lean 4.34.0 and Lake 5.0.0. The branch commit checked was

`af98b262699f4d19712682fc9a0cffe2ed353217`

Phase	Wall time	Result
Accepted base through `Comparison19`	19.25 s	success
New definition modules through `Contender20`	0.84 s	success
New proof module `Comparison20`	0.35 s	success
Focused axiom and closure audit	1.04 s (approximately)	success

The repository's contest budget permits earlier accepted modules to be precompiled. Under that convention, the relevant new phases are 0.84 seconds and 0.35 seconds, comfortably below the 15-second definition and 60-second proof limits. Timing depends on runner load and hardware, so the receipt is an observed build record, not a universal bound.

12 Repository integration

The overlay consists of four library files and one focused audit:

Path	Purpose
`BusyLean/TreeHyperReflection.lean`	Defines `spliceLifts` and `wDiagonalLift`.
`BusyLean/TreeHyperReflectionFacts.lean`	Proves progressivity, strict inflation, and child-rank strictness.
`BusyLean/Contender20.lean`	Defines `family19`, `schedule20`, `rank20`, `F20`, and `contender_20`.
`BusyLean/Comparison20.lean`	Proves pointwise, program, and final contender comparisons.
`test/Contender20Audit.lean`	Prints axioms, checks the forbidden-constant closure, and verifies theorem signatures.

After copying the files into a checkout at the audited base commit, append the four module imports to `BusyLean.lean`. Then run:

```
lake build BusyLean.Contender20
lake build BusyLean.Comparison20
lake env lean test/Contender20Audit.lean
```

The archive includes an integration patch and detailed instructions.

13 Limitations and research integrity

1. **No numerical evaluation.** The final natural number is executable in principle but not practically evaluable. The development checks its definition and comparison without reducing it.

2. **No exact ordinal assignment.** The words "large Veblen," "autonomous," and "hyper-closure" describe repository syntax and its intended proof-theoretic motivation. This work does not prove an isomorphism with a canonical ordinal notation system or identify the exact order type of `rank20`.

3. **Strict dominance is exact; the size calibration is not.** The theorem $C_{19} < C_{20}$ and the pointwise family theorem are formal. Statements about exponentiation, fixed points, or transfinite hierarchy levels are explanatory heuristics unless explicitly represented by a Lean theorem.

4. **The W-rank is external syntax.** The rank tree is constructed in Lean from autonomous tree operations. A still stronger direction would let the reflected object language generate and inspect its own rank trees, followed by a normalization proof for that enriched language.

14 Further research questions

1. **Finite-rank embedding.** Define a canonical embedding of the existing natural-rank hierarchy `diagonalLift` into `wDiagonalLift` and prove an explicit domination theorem at every finite rank, not merely strict parent-over-child growth.

2. **Rank monotonicity under tree embedding.** Identify a simulation relation $R \preccurlyeq R'$ between rank trees strong enough to prove $W_S(R)$ below $W_S(R')$ without assuming oracle monotonicity. The likely formulation must carry explicit coordinate witnesses.

3. **Generic well-founded rank types.** Replace `WTree Nat` by an arbitrary inductive well-founded branching signature and prove the amplifier theorems once, parametrically.

4. **Self-generated rank trees.** Extend the large-Veblen ladder language with a safe tree datatype and recursor, then make the rank range over every tree generated by the language itself. The challenge is to retain a transparent strong-normalization proof.

5. **Semantic ordinal calibration.** Construct an ordinal interpretation for the transformer-lift hierarchy and prove upper and lower bounds for finite and W-ranked hyperreflection. Even a coarse sandwich theorem would turn the present heuristic comparison into a mathematical classification.

6. **Reflection over rank-indexed languages.** Let each rank node select not only a lift but also a stronger universe ladder or atom notation system. This would combine transfinite amplifier rank with transfinite language strength rather than holding the language fixed.

7. **Independent kernel checking.** Add `leanchecker` and `nanoda` jobs, subject to their support for Lean 4.34.0 and the project's axiom policy, to supplement the standard Lean kernel build.

8. **Machine-checked budget harness.** Extend `scripts/check.py` so Contender 20's definition and proof phases are timed separately, the closure audit is mandatory, and a signed or hashed receipt is emitted automatically.

15 Conclusion

The proposed Contender 20 makes a specific qualitative move: the rank of BusyLean's strongest generic amplifier is promoted from a natural number to an arbitrary well-founded, countably branching tree. The node operation is not a metaphorical limit. It explicitly splices all child-rank lifts, uses the splice to amplify the base, and traverses the local schedule with the same lower-rank lift. Lean proves that every rank is progressive and strictly inflationary and that each visible child rank is strictly below its parent.

Instantiating this amplifier with the large-Veblen ladder, seeding it with the whole Contender 19 family, and choosing a self-fed autonomous W-rank yields a closed computable natural number. The checked theorem

$$\boxed{C_{19} < C_{20}}$$

meets the contest's exact success criterion, while the W-valued rank supplies a substantial structural margin beyond the previous finite-rank construction.

A Lean theorem inventory

The main declarations checked in the package are:

Declaration	Content
`spliceLifts_preservesProgressive`	A splice of progressivity-preserving lifts preserves progressivity.
`spliceLifts_strictlyInflationary`	A splice of strictly inflationary lifts is strictly inflationary.
`wDiagonalLift_preservesProgressive`	Every W-ranked lift preserves progressivity.
`wDiagonalLift_strictlyInflationary`	Every W-ranked lift strictly exceeds its base transformer.
`wDiagonalLift_child_lt`	Every visible child-rank output is strictly below the parent-rank output.
`family19_lt_F20`	$\mathrm{family}_{19}(n) < F_{20}(n)$ for every n.
`raw_reflection_lt_F20`	One raw large-Veblen reflection over the new seed is below $F_{20}(n)$.
`lvalue_lt_F20`	Every enumerated bounded large-Veblen-ladder program over the new seed is below $F_{20}(n)$.
`F19_lt_F20`	$F_{19}(n) < F_{20}(n)$ for every n.
`input_lt_F20`	$n < F_{20}(n)$ for every n.
`contender_19_lt_contender_20`	The required strict comparison.

B Core source excerpt

```
def spliceLifts (lifts : Nat -> TransformerLift) :
    TransformerLift :=
  fun base => spliceTransformers (fun i => lifts i base)

def wDiagonalLift (schedule : WTree Nat) :
    WTree Nat -> TransformerLift
  | .leaf => liftTransformer
  | .node f =>
      let lower : TransformerLift :=
        spliceLifts
          (fun i => wDiagonalLift schedule (f i))
      fun base =>
        stageLift lower (lower base) schedule

def family19 (n : Nat) : Nat := F19 (F19 n)

def schedule20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family19 n + 42)
      (Autonomous.encode (schedule19 n)))

def rank20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family19 n + 42)
      (Autonomous.encode (schedule20 n)))

def F20 (n : Nat) : Nat :=
  wDiagonalLift (schedule20 n) (rank20 n)
```

```
    lTransformer (fun _ => family19) n n

def contender_20 : Nat := F20 (F20 42)
```

C Validation receipt summary

```
commit: af98b262699f4d19712682fc9a0cffe2ed353217
Lean 4.34.0, Lake 5.0.0
accepted base: 19.25 s
definition phase: 0.84 s
proof phase: 0.35 s
contender_20 closure: 4186 constants; avoids 15 forbidden names
required theorem: contender_19 < contender_20
```

The archive contains the complete machine-generated `validation-ci.log`.

References

[1] Scott Aaronson. *Who Can Name the Bigger Number?* Online essay, 1999.

[2] Cody Roux. *name-the-biggest-number*. Coq contest repository, GitHub.

[3] Vladimir Reshetnikov. *BusyLean: name the biggest number, in Lean*. GitHub repository, audited at commit `64d31a0cf6f1aa4477a088be5554957de6e87a61`, October 13, 2026.

[4] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, Lecture Notes in Computer Science 12699, pages 625–635. Springer, 2021.

[5] Jean-Yves Girard, Paul Taylor, and Yves Lafont. *Proofs and Types*. Cambridge Tracts in Theoretical Computer Science 7. Cambridge University Press, 1989.

[6] Kurt Schuette. *Proof Theory*. Grundlehren der mathematischen Wissenschaften 225. Springer, 1977.

[7] Wolfram Pohlers. *Proof Theory: The First Step into Impredicativity*. Universitext. Springer, 2009.

[8] Ieke Moerdijk and Erik Palmgren. Wellfounded trees in categories. *Annals of Pure and Applied Logic*, 104(1–3):189–218, 2000.